



Carrera: Ingeniería de Software con IA

# MÓDULOS Y PAQUETES PARA MACHINE LEARNING CON PYTHON



## RESULTADO DE APRENDIZAJE:

Estudiante entienda qué es el Deep Learning, cómo aprende una red neuronal a nivel conceptual, y sea capaz de reconocer sus aplicaciones, implicancias éticas y las herramientas (TensorFlow/Keras) para implementarlo

## CONTENIDO:

Deep Learning  
jerarquía conceptual  
Redes neuronales  
funciones de  
forward  
NLP

# ¿Cómo aprende una computadora?

*Programar = A través de reglas.*

*Entrenar = con ejemplos y dejar que el sistema encuentre el patrón.*

Reconocer un gato en una foto es casi imposible de programar con reglas explícitas pero un humano lo hace sin esfuerzo. Ahí empieza todo.

## Preguntas para plantear

- ¿Qué diferencia hay entre dar una regla y dar ejemplos?
- ¿Qué tarea sería casi imposible resolver con reglas explícitas, pero un humano hace sin esfuerzo?

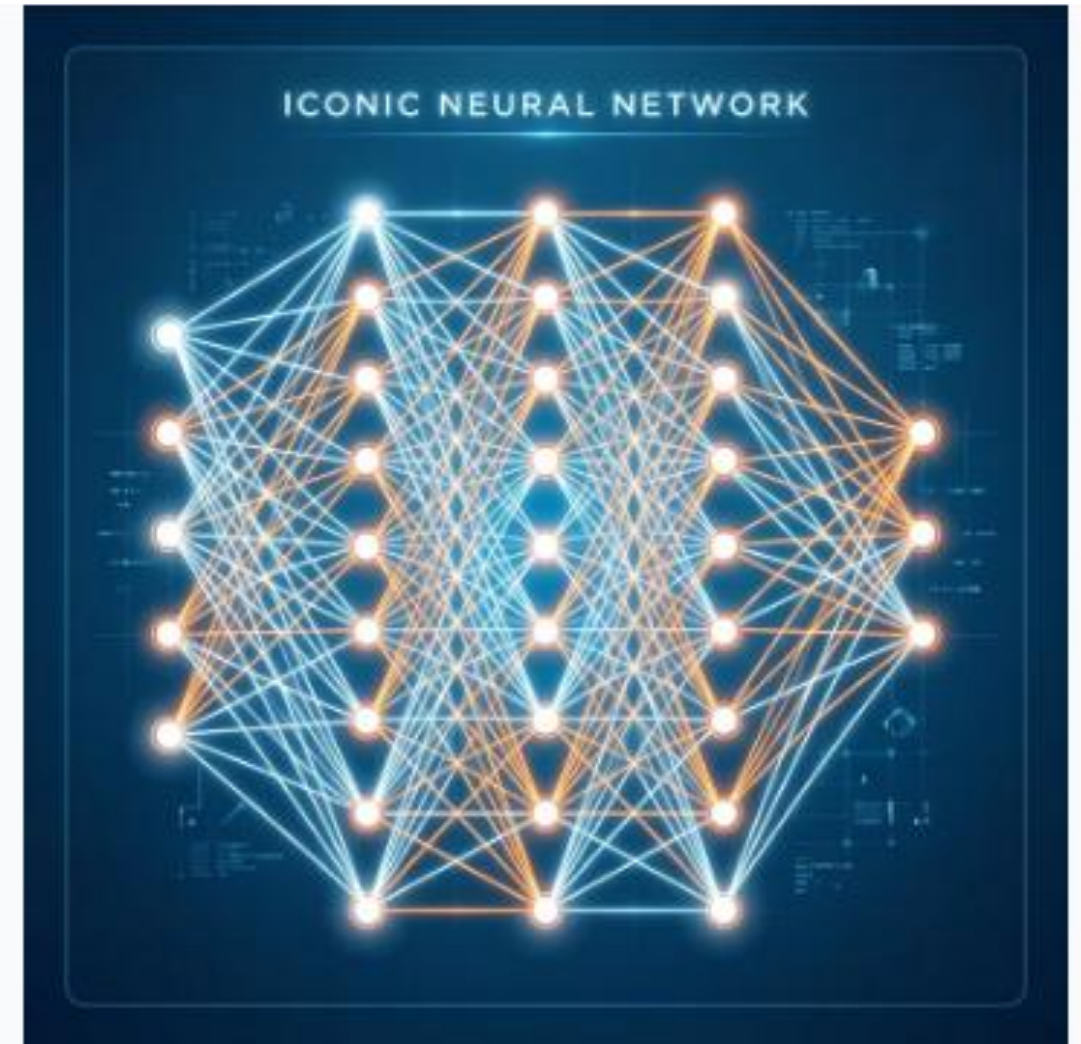
# Programación Tradicional vs ML

En el enfoque **tradicional**, ingresamos datos y reglas explícitas para obtener un resultado. En **Machine Learning**, la computadora descubre las reglas a partir de datos y resultados.

**Paradigma Tradicional:** Datos + Reglas explicitas = Resultado deseado.

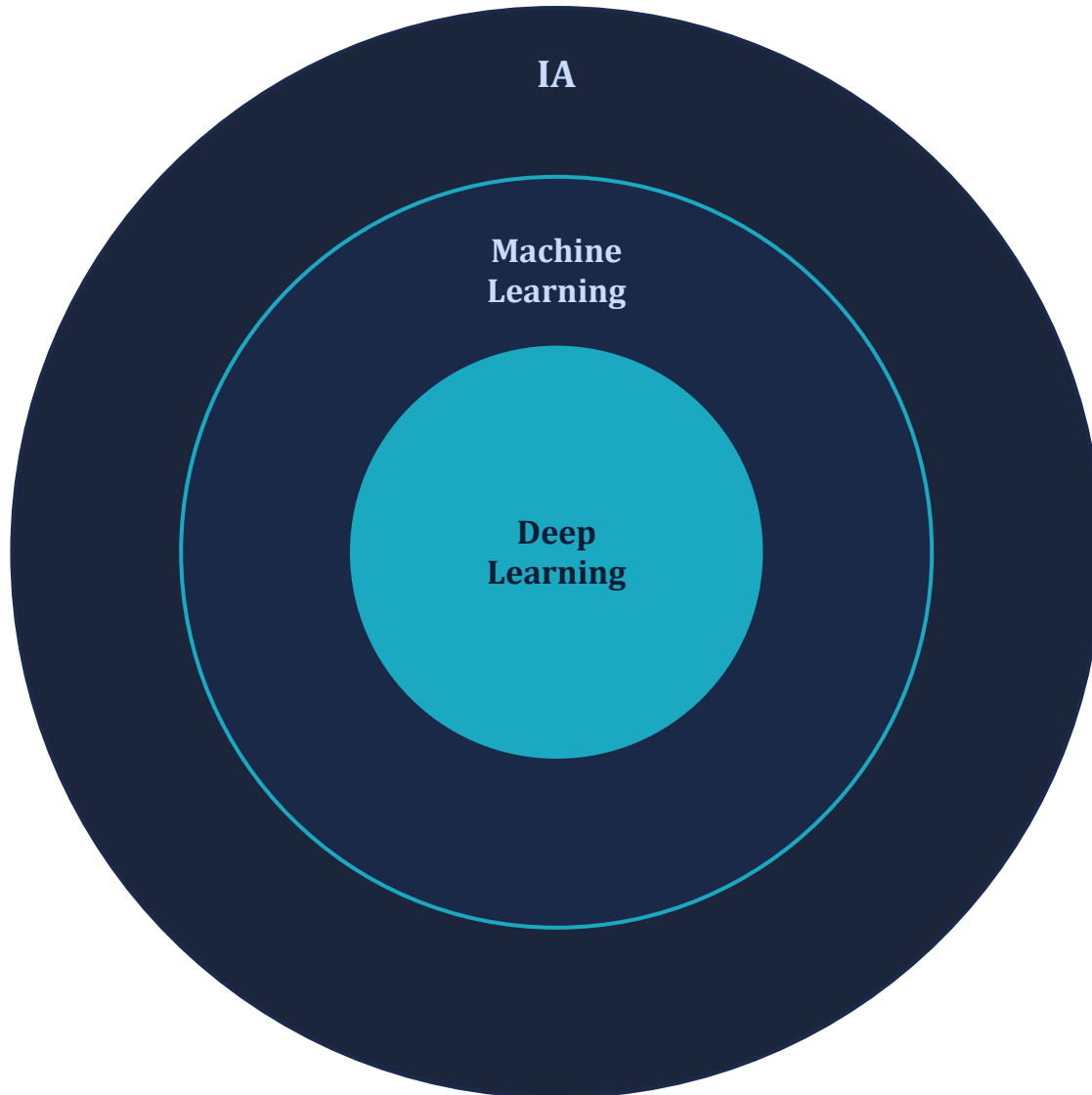
**Machine Learning:** Datos + Respuestas = Algoritmo aprende Patrones.

**Caso Práctico:** Reconocimiento de dígitos (MNIST) mediante miles de ejemplos en lugar de reglas manuales.



# IA, ML y Deep Learning

*No son sinónimos: son círculos concéntricos, cada uno contenido en el anterior.*



## Inteligencia Artificial

Cualquier técnica que imita comportamiento inteligente. Incluye sistemas basados en reglas, sin aprendizaje automático.

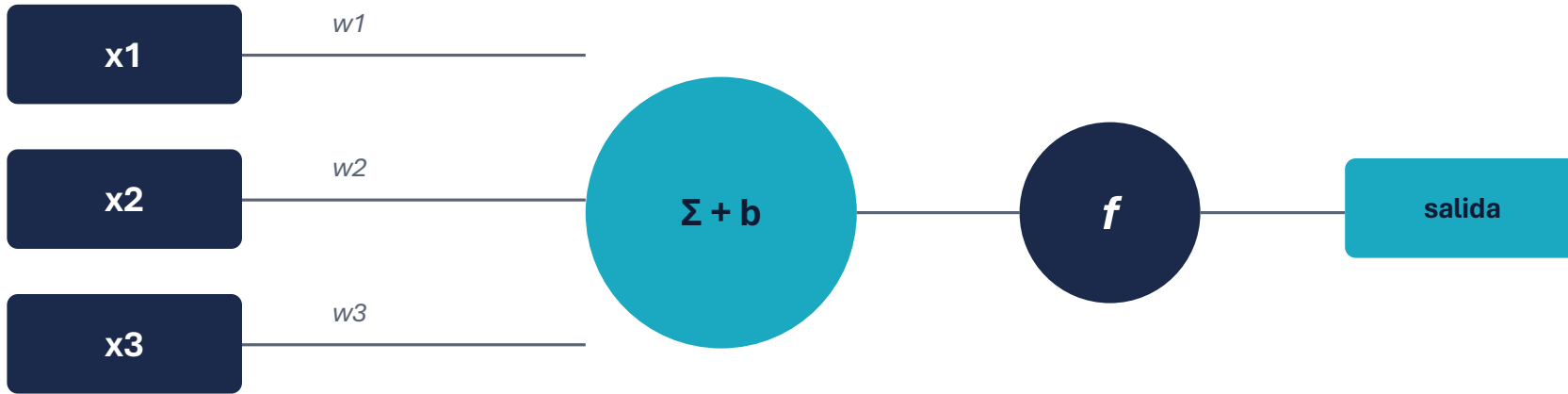
## Machine Learning

El sistema infiere reglas a partir de datos. Requiere que un humano diseñe manualmente las características (features).

## Deep Learning

Redes con múltiples capas que aprenden automáticamente qué características son relevantes, sin diseño manual.

# Redes neuronales artificiales



$$salida = f( w_1x_1 + w_2x_2 + w_3x_3 + b )$$

## Cuidado con la analogía biológica

*La neurona artificial no “dispara” como una biológica ni tiene plasticidad sináptica real. Es una inspiración histórica del nombre, no una equivalencia funcional.*

## Funciones de activación

- ReLU — la más usada en capas ocultas
- Sigmoide — comprime entre 0 y 1
- Softmax — probabilidades en la capa de salida (clasificación)

# Redes neuronales artificiales: el Perceptrón y los Pesos

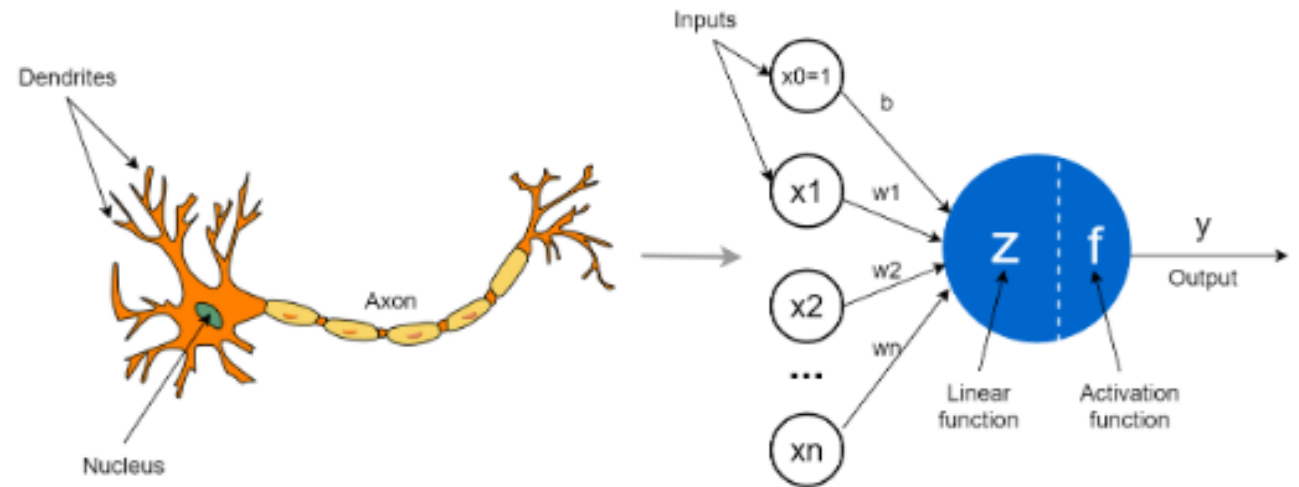
## La neurona artificial

La unidad básica procesa entradas ( ) multiplicadas por pesos ( ) y añade un sesgo ( ).

$$z = \sum_{i=1}^n w_i x_i + b \rightarrow a = f(z)$$

**Pesos (w):** Regulan la importancia de cada entrada.

**Sesgo (b):** Permite desplazar la función de activación.



# Parámetros vs Hiperparámetros

## Parámetros del modelo

Son las variables internas que la red ajusta y aprende automáticamente durante el proceso de entrenamiento.

- **Pesos ( $w$ ):** Determinan la fuerza de las conexiones.
- **Sesgos ( $b$ ):** Ajustes de desplazamiento por neurona.
- Modificados mediante optimización e iteraciones.

## Hiperparámetros

Configuraciones definidas externamente por el desarrollador antes de iniciar el entrenamiento del modelo.

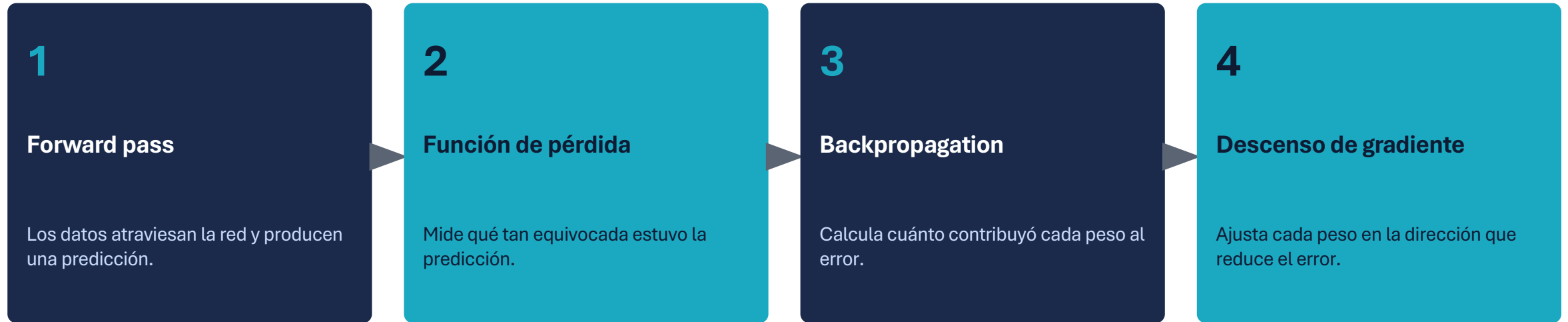
- **Tasa de aprendizaje ( $\eta$ ):** Tamaño del paso.
- **Batch Size & Epochs:** Tamaño de lote y épocas.
- **Arquitectura:** Número de capas y neuronas.



# Funciones de activación

-  No-linealidad Esencial: Sin funciones de activación, una red multicapa se comportaría matemáticamente como una simple transformación lineal, limitando su capacidad expresiva.
-  ReLU (Rectified Linear Unit): Definida como  $f(x) = \max(0, x)$ . Es la función más utilizada en las capas ocultas por su eficiencia computacional.
-  Sigmoid: Transforma la salida al rango entre 0 y 1 mediante  $f(x) = \frac{1}{1 + e^{-x}}$ , ideal para clasificación binaria.
-  Softmax: Genera una distribución de probabilidad multiclase en la capa de salida, donde todas las probabilidades suman 1.

# ¿Cómo aprende una red?



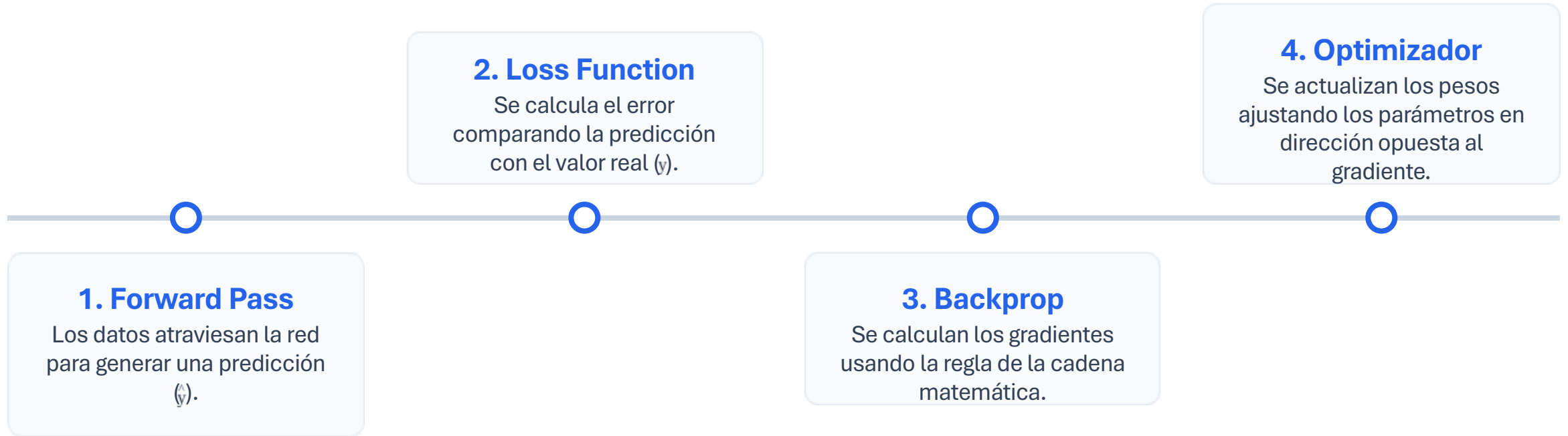
## Analogía: la montaña con niebla

*Bajas el valle sin ver, solo sintiendo la inclinación bajo tus pies. El tamaño del paso es la tasa de aprendizaje: muy grande y rebotas sin converger; muy pequeña y tardas una eternidad.*

## Cuidado

*El gradiente no siempre halla el mínimo global — hay mínimos locales. No confundas "época" (todo el dataset) con "iteración" (un batch).*

# Ciclo de entrenamiento



# Gestión de lotes y épocas

# 100

Iteraciones por Época

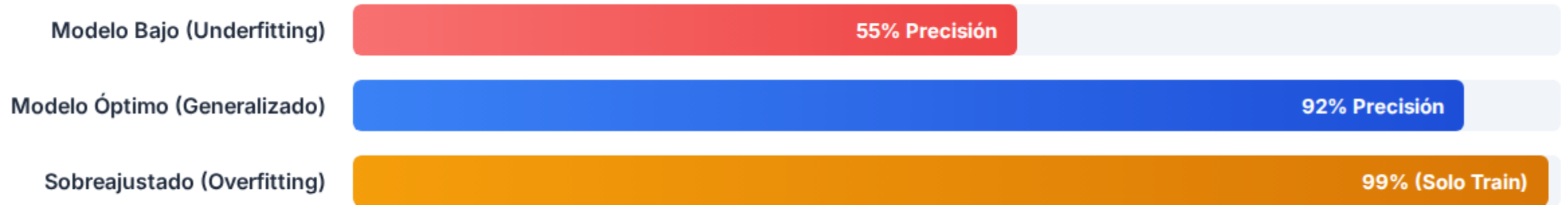
## Definición

**Epoch:** Una pasada completa de todo el conjunto de datos por la red neuronal.

**Batch Size:** Subconjunto de muestras procesadas antes de actualizar los pesos.

**Ejemplo práctico:** Para un dataset de 10,000 imágenes con un `batch_size = 100`, se ejecutan exactamente 100 iteraciones por cada época de entrenamiento.

# Desempeño y generalización



# Arquitecturas principales

| Arquitectura                       | Especialidad                      | Casos de Uso Principales                      |
|------------------------------------|-----------------------------------|-----------------------------------------------|
| <b>MLP</b> (Perceptrón Multicapa)  | Datos tabulares y densos          | Clasificación estándar, regresión de datos.   |
| <b>CNN</b> (Redes Convolucionales) | Procesamiento espacial e imágenes | Visión por computadora, detección de objetos. |
| <b>RNN / LSTM</b> (Recurrentes)    | Datos secuenciales y temporales   | Series de tiempo, análisis de audio.          |
| <b>Transformers</b>                | Mecanismos de atención paralela   | Modelos de lenguaje (LLMs), traducción.       |

# Aplicaciones del Deep Learning

## Visión

Diagnóstico médico, autos autónomos, reconocimiento facial

## NLP

Traducción, asistentes virtuales, modelos de lenguaje

## Generativa

Imagen, texto, música y video generados

## Recomendación

Streaming y e-commerce personalizados

## Ética

*Como ingenieros, tomarán decisiones de diseño con consecuencias reales sobre personas reales.*

### Sesgo algorítmico

Datos históricos desiguales → decisiones amplificadas a escala.

### Privacidad

Modelos entrenados con datos sin consentimiento informado.

### Explicabilidad

Cajas negras difíciles de justificar en salud, justicia, crédito.

### Impacto ambiental y laboral

Costo energético del entrenamiento y desplazamiento laboral.

# TensorFlow

*Framework de código abierto de Google Brain (2015) para construir y entrenar modelos de ML/DL a gran escala.*

## Tensores

Estructura de datos fundamental: generalizan escalares, vectores y matrices a n dimensiones.

## Grafo de cómputo

Operaciones como nodos y tensores como aristas. Desde TF 2.x, ejecución "eager" por defecto.

## Autograd

Diferenciación automática: calcula todos los gradientes sin derivar fórmulas a mano.

## Ecosistema

TF Lite (móvil), TF.js (navegador), TF Serving (producción) — de entrenamiento a despliegue.

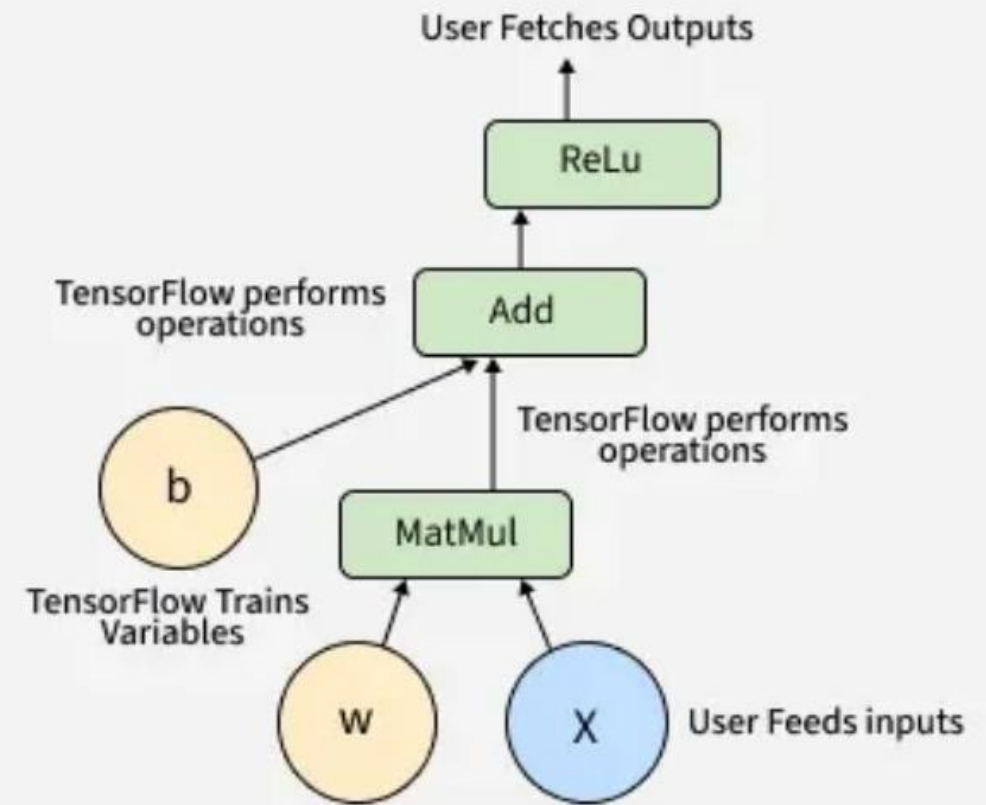


# TensorFlow

**TensorFlow:** Plataforma de código abierto desarrollada por Google para cómputo numérico de alto rendimiento en CPU y GPU.

**Keras:** API de alto nivel integrada en TensorFlow enfocada en la facilidad de uso, modularidad y rápida experimentación para prototipar modelos deep learning.

## TensorFlow Graph



# Keras

API de alto nivel creada por François Chollet, **diseñada para minimizar el tiempo entre idea y experimento.**

## Sequential

Pila lineal de capas, una tras otra. La forma más rápida de expresar una red estándar la que se usa.

## Functional

Múltiples entradas/salidas, ramas o conexiones no lineales, por ejemplo arquitecturas con conexiones residuales.

## Flujo de trabajo típico



# TensorFlow vs Keras

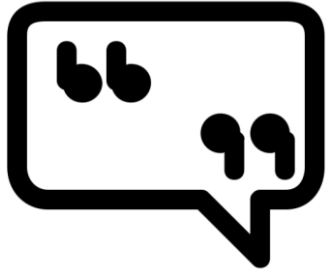
*No compiten. Desde TF 2.0, Keras (tf.keras) es la API oficial de alto nivel integrada en TensorFlow.*

## TensorFlow = el motor

Maneja tensores, cómputo en GPU, diferenciación automática y despliegue en producción.

## Keras = la interfaz

Hace ese motor accesible con pocas líneas de código legible. Bajamos a TF puro solo para arquitecturas muy no estándar o bucles de entrenamiento personalizados.



*“Los sistemas de información no solo mueven datos... mueven ideas, decisiones y el futuro de las organizaciones”*